# Florence: Architecture - LLM Engine Service

## Overview

The Florence Large Language Model Engine Service is a dedicated microservice built with FastAPI and Python 3.13. It provides artificial intelligence features to support the Florence platform. The system uses LangChain to orchestrate language models for tasks such as food analysis, biometric report parsing, health recommendation generation and clinical risk assessment.

## Architecture

The application routes are divided into feature-specific modules. The service does not connect directly to a database. It executes requests against the core data service to retrieve patient context, settings and clinical thresholds. It implements a factory pattern to instantiate language model clients dynamically. The system handles image processing for vision models and strictly formats outputs using predefined data schemas.

## Environment Configuration

The system requires specific environment variables to authenticate with the language model provider and communicate with the core data service. The developer must supply these values in a local configuration file or via the deployment platform.

- `LLM_API_KEY` : The authentication key for the language model provider.
- `LLM_BASE_URL` : The provider endpoint.
- `LLM_MODEL` : The specific model identifier to utilise.
- `LLM_TEMPERATURE` : The generation parameter controlling response variability.
- `LLM_MAX_TOKENS` : The maximum output token limit.
- `DATA_SERVICE_URL` : The uniform resource locator for the primary backend data service.

## Directory Structure

The repository contains structured directories that separate the application features.

- `main.py` : The application gateway that initialises the server and registers the routers.
- `config.py` : The environment configuration module.
- `core/` : A directory containing the language model factory and the data service client.
- `features/` : A directory housing the domain-specific business logic, routing and data models for all artificial intelligence capabilities.
- `pyproject.toml` : The configuration file that defines project metadata and the necessary Python dependencies.
- `vercel.json` : The deployment configuration file that instructs the hosting platform on how to route incoming requests.

## Package Management with uv

The project uses a standard Python configuration file but the architecture is optimised for the `uv` package manager. The `uv` tool rapidly resolves and installs dependencies. The developer can utilise it to synchronise the project environment directly from the configuration file. This ensures all required packages such as FastAPI, LangChain and HTTPX are installed consistently across different development machines.

## Installation Instructions

The developer must follow these steps to set up the local environment.

1. Install the `uv` package manager on the host system.
2. Execute the appropriate `uv` command to create a virtual environment and install the listed dependencies.
3. Populate the local environment file in the project root with the required application keys.
4. Start the application using the Uvicorn server gateway interface.

# API Modules

## General

- `GET /` : Root endpoint with service information.

## Nutrition Module

The nutrition module processes meal imagery.

- `POST /nutrition/analyze` : Accepts an image file of a meal. The system uses a vision model to analyse the food, estimate total calories and generate a brief description.

## Recommendations Module

The recommendations module generates personalised health guidance.

- `POST /recommendations/generate` : Evaluates a clinical summary against personalised thresholds to generate actionable guidance across categories such as meals, activity and sleep.
- `POST /recommendations/unified-daily` : Processes a daily health snapshot to simultaneously calculate clinical risk levels, patient insights and tactical recommendations.

## Activity Module

The activity module assesses physical exertion.

- `POST /activity/estimate-calories` : Calculates estimated calories burned based on activity duration, task description and patient biometrics.

## Biometrics Module

The biometrics module automates data entry for medical documents.

- `POST /biometrics/parse-lab-report` : Accepts an image or document containing a laboratory report. The system extracts specific values for lipid panels or blood glucose tests. It automatically standardises the extracted units to match the application expectations.

## Insights Module

The insights module provides patient encouragement.

- `POST /insights/generate` : Synthesises recent patient data and active recommendations into a single cohesive sentence for the patient dashboard.

## Simulator Module

The simulator module creates test data.

- `POST /simulator/generate` : Constructs realistic synthetic patient data sets based on a requested scenario. The generated data includes daily meals, activity logs and historical biometric readings.

## Risk Module

The risk module supports clinician decision making.

- `POST /risk/assess` : Evaluates recent health readings against personalised clinical thresholds to assign a risk tier. The system generates a clinical rationale and directly updates the patient profile in the primary data service.